Assumptions & Decisions Log

Documented per the brief: "If something is ambiguous, make a sensible assumption, write it down and carry on."

1. **Cut-off.** Settlement cut-off is 2026-06-30 23:59:59 IST (+05:30), exactly as the brief states. Configurable via `--cutoff` (ISO-8601) for reproducing historical runs.
2. **EXPIRED.** An order whose **latest** authorization is strictly more than 7 days before the cut-off, with no capture, is `EXPIRED`. Exactly 7 days → still `AUTHORIZED`. Microsecond precision comparison; "more than 7 days" is strict.
3. **Naive timestamps are IST.** All operations are Indian; the cut-off is defined in IST. Naive `sent_at` values are read as IST. Aware values (`Z`, `+00:00`, `+05:30`, …) are normalized to IST before comparison. Mixed representations of the same instant order identically (tested).
4. **Event-time ordering.** Events are applied ordered by `(sent_at_us, event_id)`, not arrival order. Ties on identical `sent_at` are broken by `event_id` for determinism — such ties carry no semantic difference unless payloads contradict, in which case source-of-truth ordering is genuinely unknowable from the log.
5. **De-duplication key is `event_id`.** Per the gateway contract, `event_id` is unique per event and delivery is at-least-once. Redeliveries byte-identical or not (e.g. different `delivery_attempt`) count as one. `delivery_attempt` is recorded but never used for semantics.
6. **Classification order.** Each raw line is: parsed → deduplicated (`event_id`) → classified as orphan (order unknown) or valid. A duplicate of an orphan counts as a duplicate, not a second orphan. Unique events = valid + orphans; `processed_events = unique − orphans`.
7. **Orphans are reported, never processed.** Unique orphan events are written to `orphan_events.csv` (with the raw amount column preserved) and excluded from totals, state machine and fees. Duplicates of orphan deliveries increment the duplicate counter only.
8. **Invalid events.** Malformed JSON, missing/blank `event_id` or `order_id`, unknown `type`, unknown/missing `api_version`, negative amounts, unparseable `sent_at`, and blank lines are counted as `invalid_events`, skipped, and the first 20 examples are kept in `summary.json` `diagnostics`. They are never silently dropped.
9. **API v1 → paise** with `Decimal`, half-up to the nearest paisa (e.g. `"100.005"` → 10001 paise). **API v2** amounts must be whole paise integers; fractional paise strings are rejected as invalid.
10. **Fee.** 2% of each captured amount, rounded to the nearest paisa, **round-half-up** (integer math: `(2·a + 50) // 100`). Computed per capture event. Fees are not refunded to the merchant when a capture is later refunded (no gateway documentation says otherwise). Documented because the alternative (half-even) differs at exact 0.5-paisa boundaries.
11. **Net settlement** = Σ captured − Σ fees − Σ refunded(processed) — i.e. what Finance should expect to land after the gateway nets refunds. `summary.json` additionally reports `net_settlement_before_refunds_inr` (Σ captured − Σ fees) for gateways/banks that settle refunds separately instead of netting them off. All arithmetic in integer paise; formatting to ₹ happens at the edges only. If your gateway's statement uses the second convention, use that figure — both are produced from the same exact paise totals.
12. **Mismatch.** Flagged true iff the order has ≥ 1 capture event **and** total captured ≠ `orders.csv` `amount_inr`. State is kept as computed (per the brief). Orders with zero captures are never flagged, regardless of amounts.
13. **Refund state.** `refund.created` = pending, counts nothing financial. Only `refund.processed` moves money. `PARTIALLY_REFUNDED` if Σ processed > 0 and < captured; `REFUNDED` if ≥ captured (over-refunds surface as REFUNDED and are counted in `diagnostics.over_refunded_orders`).
14. **Refunds without a capture** accumulate into `total_refunded_inr` and net settlement (money did leave), the order's state is computed from its auth/failure/capture events as usual, and the order is counted in `diagnostics.refunds_without_capture`. They never invent CAPTURED state.

15. **Stale failures/authorizations.** A failure with `sent_at` after the capture (or after the latest authorization has led to a capture) is ignored and counted in diagnostics. An authorization after a capture is ignored similarly. Event-time ordering decides; arrival order is irrelevant.
16. **Multiple captures per order** are summed (partial captures are a real gateway pattern). Fees apply per capture event.
17. **Failed-then-authorized.** Authorization/capture overrides an earlier failure — final state is never FAILED once an authorization exists. FAILED requires that only failed attempts exist.
18. **`amount_inr` in orders.csv** is read as an exact rupee decimal and converted with half-up paise quantization; `utf-8-sig` BOM tolerated; CRLF tolerated (`csv` module).
19. **Duplicate orders.csv rows.** First occurrence wins; duplicates are counted in `diagnostics.duplicate_orders_in_orders_csv`. This is surfaced, not silently absorbed.
20. **Amounts in outputs** are exact decimal strings with two decimals (`1234.56`). Counts are integers. `summary.json` carries machine-checkable strings; no floats anywhere in outputs.
21. **Memory model.** Input is streamed; orders and unique events are staged in an on-disk SQLite scratch file (`.reconcile_work.sqlite`, deleted after the run unless `--keep-work-db`); the final pass is a sorted merge-join. Peak RSS is O(batch), demonstrated ~20 MB on a 520k-event run — flat versus event count. For the full 50M file, expect single-digit-GB-per-hour-class runtime dominated by JSON parsing and dedupe index I/O (measured ≈11 s per 520k events on one core ⇒ ≈3 min per 10M, ≈18 min for 50M, single process; embarrassingly parallel by splitting on `order_id` hash shards if needed).
22. **SQLite as staging** here is deliberate: stdlib-only, zero services, transactional, and fast enough; for 10× the volume, swap the staging layer for Postgres or a sorted-parquet spill — the merge-join contract is unchanged.
23. **`answers.csv`** is yours to fill; we ship `payment_reconciliation ... digest` which prints every headline figure it contains so you can transcribe them without misreading JSON.
24. **Nothing here is validated against your live gateway.** Where the brief is silent, the safer reading wins (e.g. orphan isolation, invalid-line quarantine) and is logged, never guessed silently.